

MULTIPLE OUTCOMES WITH ELIF

Block 14 • Ordered choices, range categories,
and boundary testing

```
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
elif score >= 70:  
    grade = "C"  
else:  
    grade = "F"
```

A

B

C

F

Today's big rule: Python takes the first true branch, then skips the rest.

From two paths to many outcomes

Before: if / else

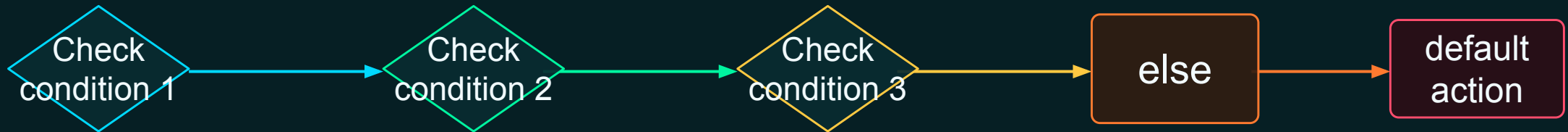
One condition.
Two possible blocks.
Exactly one branch runs.

Now: if / elif / else

Many conditions.
Many categories.
Still only one selected branch.

Mental model

A checklist that stops at the first match, not the “best” match.



Many outcomes, one execution path.

The elif chain has a predictable shape

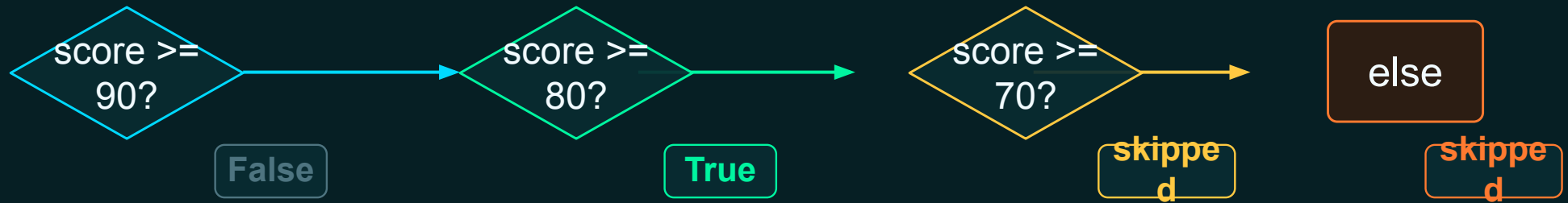
```
if condition_one:  
    action_one  
elif condition_two:  
    action_two  
elif condition_three:  
    action_three  
else:  
    default_action
```

What Python does

- Evaluate conditions from top to bottom.
- When one condition is True, run its block.
- Skip every remaining elif and else.
- Use else as the safety net when nothing matched.

An elif chain is ordered. Order is part of the program logic.

First true branch wins



score = 86

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
```

Trace it

86 is not at least 90.
86 is at least 80.
Python assigns grade = "B".
The C and F checks never run.

Score classification: descending thresholds

```
score = 86
```

```
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
elif score >= 70:  
    grade = "C"  
elif score >= 60:  
    grade = "D"  
else:  
    grade = "F"  
  
print(grade)
```

Score range	Branch
90 and above	A
80 through 89	B
70 through 79	C
60 through 69	D
below 60	F

For “at least” thresholds, high-to-low is usually the clearest order.

Order matters: Python does not pick the best match

Incorrect ordering

```
score = 95
```

```
if score >= 60:  
    grade = "D"  
elif score >= 70:  
    grade = "C"  
elif score >= 80:  
    grade = "B"  
elif score >= 90:  
    grade = "A"
```

Result: grade becomes "D"

Why?

- 95 satisfies score ≥ 60 .
- That branch runs immediately.
- Every later branch is skipped.
- Python never asks, "Is there a better match?"

Place the most specific or most restrictive condition first.

Range categories need careful edges

```
temperature = 82

if temperature < 32:
    category = "freezing"
elif temperature < 60:
    category = "cold"
elif temperature <= 80:
    category = "normal"
else:
    category = "hot"
```

Test value	Expected category
31	freezing
32	cold
59	cold
60	normal
80	normal
81	hot

A good classifier tests values just below, at, and just above every boundary.

Gaps and overlaps are classification bugs

Gap: no branch handles 80

```
if score > 90:  
    result = "excellent"  
elif score < 70:  
    result = "needs improvement"
```

What happens to a score of 80?
Without an else, no category is assigned.

Overlap: second branch can never run

```
if temperature >= 70:  
    category = "warm"  
elif temperature >= 80:  
    category = "hot"
```

80, 90, and 100 already match the first branch.
The hot branch is unreachable.

Boundary testing catches both missing categories and unreachable branches.

Default behavior keeps programs from falling through

Use else when...

- No named category matched.
- You need a safe default.
- You want to make “unexpected” visible.
- You want every input to produce an outcome.

So sometimes “no match” is acceptable.

But beginners should usually include else while learning so the program’s behavior is easy to trace.

```
else:  
    category = "unknown"
```

Exercise 1: processing status classifier

Rules

- 100 → Complete
- 90–99.99 → Final review
- 50–89.99 → In progress
- 1–49.99 → Started
- 0 → Not started

Starter code

```
completion = 72.5

# write your if / elif / else chain

print(status)
```

Goal: assign exactly one status for every possible completion value.

Think first: which rule should be checked first?

One clean solution for processing status

```
completion = 72.5

if completion == 100:
    status = "Complete"
elif completion >= 90:
    status = "Final review"
elif completion >= 50:
    status = "In progress"
elif completion > 0:
    status = "Started"
else:
    status = "Not started"

print(status)
```

Trace completion = 72.5

100? no

>= 90? no

>= 50? yes

Status: In progress

This chain works because broader lower categories come after higher ones.

Exercise 2: storage size classifier

Thresholds

Size in bytes	Category
0	Empty
1–999,999	Small
1,000,000–99,999,999	Medium
100,000,000–999,999,999	Large
1,000,000,000+	Very large

Starter

```
file_size_bytes = 87_500_000
```

```
# classify the file size
```

```
print(size_category)
```

Test these boundary values:

```
0, 1, 999_999, 1_000_000,  
99_999_999, 100_000_000,  
1_000_000_000
```

Exercise 3: error severity

```
error_count = 6

if error_count == 0:
    severity = "Clean"
elif error_count <= 5:
    severity = "Warning"
elif error_count <= 20:
    severity = "Review required"
else:
    severity = "Rejected"

print(severity)
```

Questions to ask

- What happens at 0?
- What happens at 5?
- What happens at 6?
- What happens at 20?
- What happens at 21?

Every threshold has a “just below / equal / just above” test.

Exercise 4: age of data classifier

Suggested categories

Days since update	Category
0–7	Current
8–30	Recent
31–90	Stale
91+	Expired

Implementation pattern

```
days_since_update = 42

if days_since_update <= 7:
    age_category = "Current"
elif days_since_update <= 30:
    age_category = "Recent"
elif days_since_update <= 90:
    age_category = "Stale"
else:
    age_category = "Expired"
```

For increasing upper-bound checks, low-to-high is often easiest.

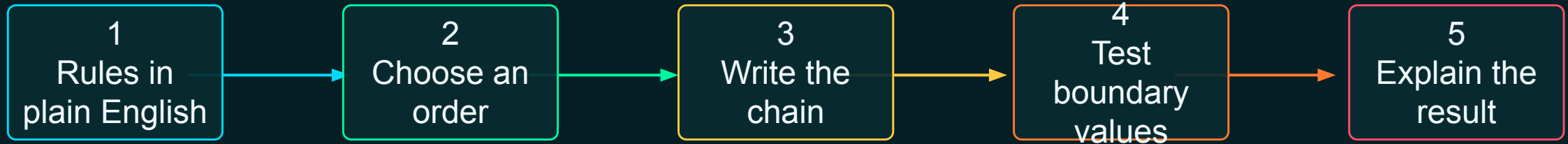
Boundary testing table

Fill this in before running your code

Threshold	Value below	At threshold	Value above	Expected categories
50	49.99	50	50.01	Started / In progress / In progress
90	89.99	90	90.01	In progress / Final review / Final review
100	99.99	100	100.01	Final review / Complete / ?

Notice the final question mark: should completion above 100 be allowed, capped, or rejected? That is a business rule.

How to build a classifier without getting lost



Instructor checkpoint: students should be able to say why a value took a specific branch, not just show that the code “worked.”

Record-quality classification

Inputs

- Completeness percentage
- Error count
- Days since last update
- Approval status

Output status

**ready • review • incomplete •
rejected**

Student deliverables

- Write rules in plain language.
- Convert rules into if / elif / else.
- Test at least six records.
- Include at least two boundary cases.
- Explain why conditions are ordered as written.

This is not just coding. It is translating policy into executable logic.

One possible rule set

Plain-language rules

- Rejected if errors are greater than 20 or data is older than 90 days.
- Incomplete if completeness is below 95%.
- Review if approval has not been received or errors are greater than 0.
- Ready only when complete enough, approved, fresh, and error-free.
- Else can be used as the ready branch when earlier problems are ruled out.

Important: your exact rules may differ. The goal is to make the rules explicit before coding.

Translate the rules into ordered code

```
if error_count > 20 or days_since_update > 90:  
    overall_status = "rejected"  
elif completion_percentage < 95:  
    overall_status = "incomplete"  
elif not approval_received or error_count > 0:  
    overall_status = "review"  
else:  
    overall_status = "ready"  
  
print(overall_status)
```

The most serious disqualifiers
Why this order?
come first.

Then incomplete records.

Then records that are nearly
usable but need review.

Only the leftovers are ready.

Order should match the meaning of the categories.

Six-record test set

Complete %	Errors	Days old	Approved?	Expected status
100	0	2	True	ready
94.9	0	2	True	incomplete
99	2	4	True	review
100	0	10	False	review
100	21	2	True	rejected
100	0	91	True	rejected

A test set should include normal cases, edge cases, and failure cases.

A quick checklist for elif bugs

Syntax

- Do all if/elif/else lines end with a colon?
- Are branch bodies indented?
- Is else last?

Logic

- Could an earlier branch steal a later category?
- Are there gaps?
- Are any branches unreachable?

Testing

- Try values below, at, and above thresholds.
- Print the final category.
- Explain the path out loud.

When an elif chain surprises you, trace it top to bottom.

Can you explain these in plain language?

- What does elif do?
- What does “first true branch wins” mean?
- Why can condition order change the result?
- What is a boundary value?
- Why does else help?

Predict the result

score = 80

```
if score > 80:
    result = "high"
elif score >= 80:
    result = "medium"
else:
    result = "low"
```

print(result)

Answer: medium

What to leave with

elif lets a program classify data into more than two outcomes.

- Python checks branches from top to bottom.
- The first true branch runs; the rest are skipped.
- Range boundaries must be tested deliberately.
- Gaps and overlaps are logic bugs, not syntax bugs.
- Clear rules lead to clear code.

Next: use decision logic inside bigger programs, then loops and collections.